Lambdas and Callables

Function pointers and functors first, then how a lambda works, captures and their lifetime traps, init-capture, generic lambdas, and when to use a function pointer, functor, lambda or `std::function`.

Before lambdas: function pointers and functors

Algorithms such as `std::sort` and `std::count_if` take "something callable". Before C++11 there were two ways to supply it, and a lambda is a shorter way to write the second.

A function pointer

```
bool descending(int a, int b) { return a > b; }
int twice(int x) { return 2 * x; }
void apply(int (*f)(int), int x) { std::cout << f(x) << '\n'; }   // a parameter that points to a function

std::vector<int> v{5, 2, 9, 1};
std::sort(v.begin(), v.end(), descending);          // 9 5 2 1: pass the function by name
apply(twice, 21);                                   // 42
```

- A function pointer is just an address (8 bytes). It can point to any function with the **same signature**, chosen at run time.
- It has **no state**: the function sees only its arguments and globals. To give `descending` a limit you would need a global variable.
- The call goes through the pointer, so the compiler usually cannot inline it.

A functor (function object)

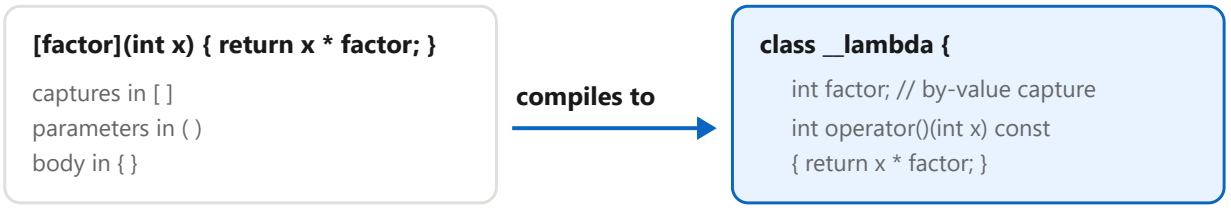
A **functor** is a class with an `operator()`. An object of it can be called like a function, and it can **carry state** in its members.

```
struct GreaterThan {
    int limit;                                          // the state
    bool operator()(int x) const { return x > limit; }
};
GreaterThan gt{4};
gt(5);                                                  // true
gt(3);                                                  // false
std::count_if(v.begin(), v.end(), GreaterThan{1});      // 3: how many elements
are above 1
```

- Each functor class is its own type, so a template like `std::count_if` knows exactly which `operator()` to call and can **inline** it.
- The cost: a whole class, written away from the place where it is used, for one small piece of logic.

> **SO WHAT IS A LAMBDA?** A lambda is a functor the compiler writes for you, in place: `[limit](int x) { return x > limit; }` is the `GreaterThan` above, with `limit` as its member. The next sections show how it works and where it goes wrong.

What a lambda is



Capture by reference stores a reference instead; it dangles if the lambda outlives the variable.

A lambda is an unnamed function object written in place: the compiler generates a small class whose constructor stores the captures and whose `operator()` holds the body.

```
//  [captures] (parameters) -> return_type { body }
auto add = [](int a, int b) { return a + b; };
add(2, 3);                                          // 5

std::vector<int> v{5, 2, 9, 1};
std::sort(v.begin(), v.end(), [](int a, int b) { return a > b; });   //
descending: an inline comparator
```

- Every lambda has its **own unique type**, so store it with `auto` (or a template parameter).
- A lambda with **no captures** converts to a plain function pointer. Compiled: `sizeof` of such a lambda is 1, of one capturing an `int` is 4, of a function pointer 8.
- `operator()` is `const` by default; `mutable` lifts that (next sections).

Captures: by value and by reference

```
int factor = 3;
auto byValue = [factor](int x) { return x * factor; };    // copies factor
now
auto byRef   = [&factor](int x) { return x * factor; };   // refers to the
live variable
auto all     = [=] (int x) { return x * factor; };        // [=] all by
value, [&] all by reference
```

> **DANGLING CAPTURES** A lambda that captures a local by reference and then outlives it (returned from the function, or run later on another thread) reads destroyed memory. When a lambda escapes its scope, capture by value.

> **[=] AND THIS** Inside a member function `[=]` also captures `this` (the pointer), so the lambda uses the object through that pointer and dangles if the object dies. This implicit capture is deprecated in C++20: write `[=, this]` to say so, or `[*this]` to copy the object.

`mutable`, init-capture, generic and recursive lambdas

```
auto count = [n = 0]() mutable { return ++n; };           // own state;
mutable lets it change
count(); count(); count();                                // 1 2 3 (compiled:
"123")

auto owned = std::make_unique<int>(42);
auto init  = [p = std::move(owned)] { return *p; };       // C++14 init-
capture: MOVE a move-only object in
                                                          // (compiled:
returns 42, the original is null)

auto bigger = [](const auto& a, const auto& b) { return a < b ? b : a; };
// C++14 generic: any type
bigger(3, 7);  bigger(2.5, 1.5);                          // 7 and 2.5

auto result = [&] { return compute(a, b); }();            // immediately
invoked: initialise a const with logic
const int limit = [&] { return fast ? 100 : 10; }();
```

- **Init-capture** `[name = expr]` creates a new member: the way to move a `unique_ptr` in, or to precompute something.
- `mutable` changes the lambda's own copy; each copy of the lambda has its own state.
- A **generic lambda** has a templated `operator()`: `auto` parameters are template parameters.
- A lambda cannot name itself. For recursion, pass it to itself: `auto fact = [](auto self, int n) -> int { return n <= 1 ? 1 : n * self(self, n - 1); };` (compiled: 5! = 120), or use `std::function`.
- `[this]` in a member function captures the pointer; `[*this]` copies the object.

Function pointer, functor, lambda, `std::function`

Four ways to pass "something callable":

KIND	CAN HOLD STATE?	INLINABLE?	OVERHEAD
function pointer	no	rarely (indirect call)	8 bytes
functor (class with `operator()`)	yes	yes: type known at compile time	size of its members
lambda	yes (captures)	yes: it is a functor	size of its captures
`std::function<R(Args...)>`	yes	no: type-erased	indirect call, and may allocate

```
template <typename F>
void apply(F f) { f(1); }                                   // template parameter: the lambda's exact type, fully inlined

void apply2(std::function<void(int)> f) { f(1); }           // type-erased: accepts anything, one indirect call

std::vector<std::function<int(int)>> ops{ [](int x) { return x + 1; },
                                          [k = 10](int x) { return x * k; }
};   // different lambdas, one type
```

- **Templates for hot paths and algorithms** (`std::sort` takes its comparator as a template parameter and inlines it). `std::function` when you must **store** callables of different types or keep them behind an interface.
- `std::function` may **allocate** when the callable does not fit its small buffer. Compiled with libstdc++: a small capture did not allocate, a 64-byte capture did.
- `std::invoke(f, args...)` calls anything callable, including a member function pointer: `std::invoke(&Account::balance, acct)`.
- A stateless lambda converts to a function pointer, handy for C APIs and callbacks.

`std::bind`: know it, prefer a lambda

```
using namespace std::placeholders;
auto f = std::bind(add3, 1, _1, _2);     // fix the first argument, leave two
open: f(2, 3) is add3(1, 2, 3)
auto g = std::bind(add3, _2, _1, 9);     // placeholders can reorder: g(4, 5)
is add3(5, 4, 9)
auto h = [](int b, int c) { return add3(1, b, c); };   // the same as f, and
easier to read
```

Compiled: `f(2, 3)` gave 123, `g(4, 5)` gave 549 and the lambda gave 123. `std::bind` predates lambdas (C++11); it copies its arguments, hides overloads and produces unreadable errors. You will see it in older code, so be able to read it; in new code write a lambda, which also inlines better.

> KEY POINT "Lambda vs `std::function`?" A lambda has its own type and is inlined; `std::function` erases the type, costs an indirect call and maybe an allocation. Use it only when you must store different callables in one place.

Further reading by section

Where each section of this lesson is explained in more depth. Pages are PDF viewer pages (the number your PDF reader shows).

SECTION	BOOK	PART OF THE BOOK	PDF PAGES
Before lambdas: function pointers and functors	C++ Primer	6.7 Pointers to functions	276 to 281
	Programming: Principles and Practice	21.1 to 21.2, 21.5 Standard algorithms, function objects, sorting and searching	617 to 626 and 633 to 635
	Effective Modern C++	Item 31, Avoid default capture modes	234 to 242
	Effective Modern C++	Item 32, Use init capture to move objects into closures	242 to 247
What a lambda is	C++ Primer	10.3 Customizing operations	414 to 429
Captures: by value and by reference	C++ Primer	10.3 Customizing operations	414 to 429
Function pointer, functor, lambda, std::function	C++ Primer	14.8 The function-call operator	600 to 607